

Como funciona la app por dentro

Recorrido por el código de Inventario iOS, siguiendo los procesos reales: arrancar, entrar, leer, guardar, sincronizar.

Ferreteria Zamora — App de gestion de inventario

UIKit + Storyboard | Core Data | URLSession con dataTask | MVVM

Como leer este documento

No esta organizado por carpetas sino por **procesos**. Cada parte sigue una accion completa del usuario desde que la dispara hasta que termina, atravesando todas las capas que hagan falta. Es la forma en que realmente se entiende un código: viendo el camino que hace un dato, no la lista de archivos.

La integracion con Firebase tiene su propio documento aparte ([Firebase-Implementacion.pdf](#)) y aca solo se menciona donde encaja.

Una sola idea para llevarse. Si tuvieras que quedarte con una frase de todo el documento: **la pantalla nunca lee de la red**. Lee de una base local, y la red es un proceso aparte que la actualiza. Todo lo demas — los campos raros de Core Data, el orden de la sincronizacion, los chips naranjas — existe para sostener esa decision.

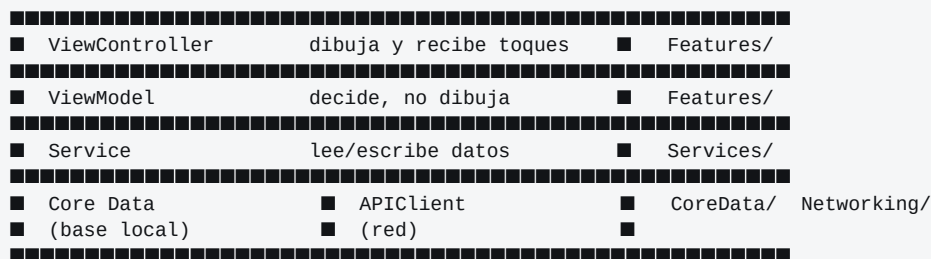
PARTE I

El esqueleto

Donde vive cada cosa y como se conectan las capas.

1. Las capas

El código se organiza en cinco capas. Una pantalla nunca salta capas: siempre habla con la de abajo.



Carpeta	Que contiene
App/	AppDelegate y SceneDelegate: el arranque
Auth/	SessionManager y KeychainService: quien esta logueado
CoreData/	El modelo .xcdatamodeld y PersistenceController

Carpeta	Que contiene
Networking/	APIClient, Endpoint, APIError, FechaAPI
Models/	DTOs: el reflejo del JSON que manda la API
Services/	Leen y escriben: unos en Core Data, otros contra la red
Sync/	SyncManager: la sincronizacion bidireccional
Features/	Una carpeta por pantalla, con su ViewController y ViewModel
DesignSystem/	Theme y estilos compartidos

Que es MVVM aca

El ViewController solo dibuja y reenvia toques. El ViewModel tiene la logica y avisa de los cambios con **closures**, no con Combine ni delegados:

Features/Productos/ProductoListViewModel.swift

```
final class ProductoListViewModel {

    var onCambio: (() -> Void)?           // "los datos cambiaron, redibuja"
    var onError: ((String) -> Void)?      // "algo fallo, muestra este mensaje"
    var onLoadingChanged: ((Bool) -> Void)? // "prendé/apagá el spinner"

    private(set) var productos: [ProductoEntity] = []
```

El ViewController se suscribe una vez en viewDidLoad y a partir de ahi solo reacciona. El private(set) significa que la pantalla puede leer la lista pero no modificarla.

2. El arranque

Cuando el usuario toca el icono de la app, esto es lo que pasa en orden:

1. AppDelegate corre didFinishLaunching: arranca Firebase.
2. PersistenceController se instancia la primera vez que alguien lo pide, y ahi carga la base local (ver abajo).
3. SessionManager se instancia leyendo el Keychain: si hay un token guardado, la sesion ya esta iniciada.
4. **El Storyboard** presenta la pantalla inicial, que es el **Login**.
5. Si SessionManager.isAuthenticated es verdadero, el Login dispara solo el segue a la app (auto-login).

PersistenceController: la base local

CoreData/PersistenceController.swift

```
final class PersistenceController {
    static let shared = PersistenceController()

    let container: NSPersistentContainer

    private init() {
        container = NSPersistentContainer(name: "InventarioModel")
        container.loadPersistentStores { _, error in
            if let error = error {
                fatalError("No se pudo cargar Core Data: \(error)")
            }
        }
        container.viewContext.automaticallyMergesChangesFromParent = true
    }

    var viewContext: NSManagedObjectContext { container.viewContext }

    func saveContext() {
        guard viewContext.hasChanges else { return }
        do { try viewContext.save() }
        catch { assertionFailure("Error guardando Core Data: \(error)") }
    }
}
```

NSPersistentContainer es el objeto de Core Data que junta el modelo, el coordinador y el contexto. El **contexto** (viewContext) es donde viven los objetos en memoria: se modifican ahí y recién con save() se escriben al disco.

El fatalError no es descuido. Si Core Data no carga, la app no tiene fuente de datos: no hay nada útil que mostrar. Arrancar igual solo lleva a un crash más confuso tres pantallas después, cuando ya no se ve la causa.

El AppDelegate guarda al mandar la app a background, por si quedo algo sin persistir:

App/SceneDelegate.swift

```
func sceneDidEnterBackground(_ scene: UIScene) {
    // Persistir cambios pendientes al mandar la app a background.
    PersistenceController.shared.saveContext()
}
```

PARTE II

Proceso 1: entrar a la app

Del botón Acceder al token guardado. Toca Login, AuthService, APIClient, Keychain y SessionManager.

3. El recorrido completo del login



AuthService: la capa mas fina de todas

Services/AuthService.swift

```

struct AuthService {
    func login(username: String, password: String,
              completion: @escaping (Result<LoginResponse, APIError>) -> Void) {
        let body = LoginRequest(username: username, password: password)
        APIClient.shared.post(.login, body: body, authenticated: false, completion: completion)
    }
}

```

Son cuatro lineas y aun asi vale la pena que exista: el `ViewModel` no sabe que el login es un POST a `/api/auth/login`. Si el backend cambiara ese contrato, se toca aca y nada mas.

El `authenticated: false` es la clave: es la unica peticion de toda la app que va sin token, porque justamente lo esta pidiendo.

4. APIClient: como se hace una peticion

Todas las llamadas al backend pasan por aca. Expone un metodo por verbo HTTP y todos terminan en el mismo lugar:

Networking/APIClient.swift

```

func get<Response: Decodable>(
    _ endpoint: Endpoint,
    authenticated: Bool = true,
    completion: @escaping (Result<Response, APIError>) -> Void
) {
    var request = URLRequest(url: endpoint.url)
    request.httpMethod = "GET"
    attachAuthIfNeeded(&request, authenticated: authenticated)
    send(request, authenticated: authenticated, completion: completion)
}

```

El es lo que permite que un solo metodo sirva para cualquier respuesta: el tipo lo decide quien llama, y Swift lo deduce del contexto. Por eso en el `SyncManager` se ve esta forma rara:

```

APIClient.shared.get(.categorias) { (resultado: Result<[CategoriaDTO], APIError>) in

```

Ese tipo escrito a mano no es adorno: es lo que le dice al compilador en que tipo hay que decodificar el JSON. Sin eso no podria deducirlo.

El token se agrega solo

```
private func attachAuthIfNeeded(_ request: inout URLRequest, authenticated: Bool) {
    // Se lee de SessionManager y no del Keychain directo: con "mantener
    // sesion iniciada" desactivado el token vive solo en memoria.
    guard authenticated, let token = SessionManager.shared.token else { return }
    request.setValue("Bearer \(token)", forHTTPHeaderField: "Authorization")
}
```

Ninguna pantalla arma cabeceras de autenticacion: es imposible olvidarse el token, y tambien imposible mandarlo a donde no va.

El manejo de la respuesta

```
guard (200...299).contains(httpResponse.statusCode) else {
    // Un 401 solo significa "sesion vencida" si la request iba con token.
    // En el login (authenticated: false) significa credenciales malas, y
    // ahi lo que sirve es el `mensaje` que manda el backend.
    if httpResponse.statusCode == 401, authenticated {
        DispatchQueue.main.async {
            SessionManager.shared.manejarSesionExpirada()
            completion(.failure(.unauthorized))
        }
        return
    }
    let mensaje = (try? JSONDecoder().decode(BackendErrorBody.self, from: data))?.mensaje
    DispatchQueue.main.async {
        completion(.failure(.server(status: httpResponse.statusCode,
                                     message: mensaje ?? "Error del servidor")))
    }
    return
}
```

El detalle fino del 401. El mismo codigo significa dos cosas distintas segun el contexto. Con token, es "tu sesion vencio" y hay que sacar al usuario. Sin token — o sea en el login — es "usuario o contrasena incorrectos", y sacarlo no tendria sentido porque todavia no entro. Por eso el authenticated del if no es opcional.

Todos los completion van dentro de `DispatchQueue.main.async`. El `dataTask` corre en un hilo de fondo, y tocar la interfaz fuera del hilo principal es una de las formas mas comunes de crashear una app iOS.

Dos respuestas especiales

```
/// Respuesta para endpoints que no devuelven cuerpo (tipicamente DELETE con
/// 204 No Content). Sin esto, `send` intentaria decodificar un body vacio y
/// fallaria con un error de decoding aunque la operacion haya salido bien.
struct RespuestaVacía: Decodable {}

/// Respuesta cuyo contenido no interesa: la operacion se confirma por el status
/// HTTP y el estado real se vuelve a bajar del servidor despues.
struct RespuestaIgnorada: Decodable {
    init(from decoder: Decoder) throws {}
}
```

RespuestaVacía resuelve un problema real: un DELETE exitoso devuelve 204 sin cuerpo, y JSONDecoder con un cuerpo vacío lanza error. La operación salió bien pero la app diría que falló.

5. Donde se guarda el token

El check "Mantener sesión iniciada" del login no es cosmético: cambia donde vive el token.

Check	Donde vive el token	Al cerrar la app
Marcado	Keychain (cifrado por iOS)	Sigue ahí: no vuelve a pedir login
Sin marcar	Solo en memoria	Se pierde: vuelve a pedir login

Auth/SessionManager.swift

```
func handleLogin(response: LoginResponse, recordarSesion: Bool) {
    token = response.token
    username = response.username
    roles = response.roles
    isAuthenticated = true

    if recordarSesion {
        keychain.saveToken(response.token)
        defaults.set(response.username, forKey: usernameKey)
        defaults.set(response.roles, forKey: rolesKey)
    } else {
        keychain.deleteToken()
        defaults.removeObject(forKey: usernameKey)
        defaults.removeObject(forKey: rolesKey)
    }
}
```

Por que Keychain y no UserDefaults. UserDefaults es un archivo de texto plano dentro de la app: cualquiera con acceso al dispositivo o a un backup puede leerlo. El **Keychain** lo cifra el sistema operativo. El token es una credencial — con el se puede operar como el usuario — así que va ahí. El nombre y los roles, que no son secretos, sí van en UserDefaults.

Notar que **la contraseña nunca se guarda**, en ningún lado. Se usa una vez para pedir el token y se descarta.

PARTE III

Proceso 2: ver el listado de productos

El caso más importante: cómo se muestra una lista sin tocar la red.

6. La regla de oro

Cuando el usuario abre la pestaña Productos, la app **no llama al servidor**. Lee de Core Data:

```

ProductoListViewController.viewDidLoad()
    ▼
viewModel.cargarLocales()
    ▼
ProductoService.productosLocales()    <- NSFetchRequest a Core Data
    ▼
onCambio?()                          <- el closure avisa a la pantalla
    ▼
tableView.reloadData()

```

La red no aparece por ningun lado. Por eso la lista funciona en el deposito sin senal, y por eso aparece instantaneamente en vez de mostrar un spinner.

7. ProductoService: la consulta

Services/ProductoService.swift

```

func productosLocales() -> [ProductoEntity] {
    let request = NSFetchRequest<ProductoEntity>(entityName: "ProductoEntity")
    request.predicate = NSPredicate(format: "pendienteEliminar == NO")
    request.sortDescriptors = [
        NSSortDescriptor(
            key: "nombre",
            ascending: true,
            selector: #selector(NSString.localizedCaseInsensitiveCompare(_:))
        )
    ]
    return (try? contexto.fetch(request)) ?? []
}

```

Tres cosas para mirar:

- NSFetchRequest es la consulta. Es el equivalente a un SELECT.
- predicate es el WHERE. Aca esconde los productos que el usuario ya borro pero que siguen locales esperando que el servidor confirme la baja.
- localizedCaseInsensitiveCompare ordena alfabeticamente ignorando mayusculas y tildes; sin ese selector, "Ácido" quedaria despues de "Zinc".

El (try? ...) ?? [] convierte cualquier error en lista vacia. Es una decision deliberada: si la consulta local falla, mostrar la lista vacia es mejor que crashear.

DTO y Entity: dos cosas distintas con el mismo nombre

Es la confusion mas comun al leer este codigo. Hay dos representaciones de un producto:

	ProductoDTO	ProductoEntity
Que es	Reflejo del JSON de la API	Fila de la base local
Donde vive	Models/Inventario/	Core Data (generado)
Campos extra	ninguno	localId, apiId, estadoSync, pendienteEliminar
Para que	Decodificar la respuesta HTTP	Todo lo que muestra la app

El SyncManager convierte de uno a otro. La pantalla **solo** ve Entity.

PARTE IV

Proceso 3: guardar un producto

Que pasa al tocar Guardar, y los cuatro campos que hacen posible el offline.

8. Los cuatro campos de sincronizacion

Cada entidad local tiene, ademas de sus datos de negocio, cuatro campos tecnicos. Entender estos cuatro campos es entender toda la sincronizacion:

Campo	Tipo	Para que sirve
localId	String	Identificador que la app genera. Existe siempre, incluso antes de que el servidor conozca el registro
apiId	Int64?	Id que asigno el servidor. Es nil mientras el registro no se haya subido nunca
estadoSync	Int16	0 = tiene cambios sin subir. 1 = igual que en el servidor
pendienteEliminar	Bool	El usuario lo borro, pero el DELETE todavia no se confirmo contra el servidor

Por que dos ids. Un producto creado sin conexion todavia no existe para el servidor, asi que no puede tener su id. Pero la app necesita identificarlo ya para mostrarlo, editarlo o borrarlo. `localId` cubre ese hueco; `apiId` se completa despues, cuando el POST devuelve el id asignado.

9. Crear

Services/ProductoService.swift

```
@discardableResult
func crear(nombre: String, precio: Double, stock: Int32,
           categoria: CategoriaEntity?, proveedor: ProveedorEntity?) -> ProductoEntity {
    let producto = ProductoEntity(context: contexto)
    producto.localId = UUID().uuidString
    // apiId queda nil: todavia no existe en el servidor. El SyncManager lo
    // completa cuando el POST devuelve el id asignado.
    producto.apiId = nil
    producto.estadoSync = 0
    producto.pendienteEliminar = false
    producto.fechaRegistro = Date()

    aplicar(nombre: nombre, precio: precio, stock: stock,
            categoria: categoria, proveedor: proveedor, a: producto)
    PersistenceController.shared.saveContext()
    return producto
}
```

Se escribe en Core Data y listo. **No hay ninguna llamada a la red.** El producto queda con `estadoSync = 0`, que es lo que dibuja el chip naranja PENDIENTE en la lista.

El detalle de aplicar()


```
private func aplicar(..., a producto: ProductoEntity) {
    producto.nombre = nombre
    producto.precio = precio
    producto.stock = stock
    producto.categoria = categoria
    producto.proveedor = proveedor
    // Marca la fila como pendiente de subir. Tambien protege el cambio: el
    // upsert de la bajada saltea todo lo que tenga estadoSync == 0.
    producto.estadoSync = 0
}
```

Ese estadoSync = 0 al final es el que hace que **editar** tambien marque pendiente. Y como dice el comentario, cumple una segunda funcion defensiva que se entiende en la Parte V.

10. Borrar: por que no se borra

```
func marcarParaEliminar(_ producto: ProductoEntity) {
    if producto.apiId == nil {
        contexto.delete(producto)
    } else {
        producto.pendienteEliminar = true
        producto.estadoSync = 0
    }
    PersistenceController.shared.saveContext()
}
```

El borrado es **logico**, no fisico. La fila se marca y sigue en la base hasta que el servidor confirme el DELETE. Para el usuario desaparece igual, porque productosLocales() la filtra.

El if apiId == nil cubre el caso del producto que nunca se subio: no hay nada que borrar del otro lado, asi que se elimina de una.

Por que no borrar y listo. Si la fila se borrara del telefono al instante y la app estuviera sin conexion, nadie recordaria que hay que avisarle al servidor. En la proxima sincronizacion el producto volveria a bajar, intacto, como si nunca lo hubieras borrado.

PARTE V

Proceso 4: sincronizar

El corazon de la app. Tres pasos, y el orden entre ellos es todo.

11. Los tres pasos

SyncManager.sincronizar() hace tres cosas, siempre en este orden:

- | | | |
|----------|--------------------------------|-----------------------------------|
| 1. SUBIR | lo que tiene estadoSync == 0 | POST (si no hay apiId) / PUT |
| ▼ | | |
| 2. BAJAS | lo que tiene pendienteEliminar | DELETE, y recien ahi borrar local |
| ▼ | | |
| 3. BAJAR | traer todo del servidor | y hacer upsert local por apiId |

El orden es la decision de diseno mas importante de la app. Si el paso 3 corriera primero, pisaria con la version del servidor los cambios locales que todavia no se subieron: se perderia todo el trabajo hecho sin conexion. Subir primero y bajar al final es lo que hace que el offline realmente funcione.

Y dentro de cada paso, tambien importa

La documentacion del propio metodo lo explica:

Sync/SyncManager.swift

```
/// - **Al subir**, categorias y proveedores van antes que productos. Un
/// producto creado sin conexion puede apuntar a una categoria tambien
/// creada sin conexion, y `ProductoRequest` solo referencia las que ya
/// tienen `apiId`. Si el producto subiera primero, se subiria sin
/// categoria y el vinculo se perderia sin ningun error visible.
/// - **Al borrar**, productos van antes. Borrar una categoria que todavia
/// tiene productos colgando es pedirle al backend un conflicto evitable.
```

"Se perderia sin ningun error visible" es lo peligroso de ese caso: no crashea, no avisa, simplemente el producto queda sin categoria y nadie se entera.

12. Como se encadenan los pasos

Siete operaciones asincronas seguidas, cortando en la primera que falle. Anidar siete `completion` era ilegible, asi que se resolvió con recursion sobre una cola:

```
private func encadenar(
    _ pasos: [(@escaping (Result<Void, APIError>) -> Void) -> Void],
    completion: @escaping (Result<Void, APIError>) -> Void
) {
    var cola = pasos
    guard !cola.isEmpty else {
        completion(.success(()))
        return
    }
    let paso = cola.removeFirst()
    paso { [weak self] resultado in
        guard let self else { return }
        switch resultado {
        case .failure(let error):
            completion(.failure(error)) // corta la cadena
        case .success:
            self.encadenar(cola, completion: completion) // sigue con el resto
        }
    }
}
```

Cada paso se saca del frente de la cola, se ejecuta, y en su respuesta se llama de nuevo con el resto. Si uno falla, se corta y no se ejecuta ninguno de los siguientes. Es, en esencia, lo que `async/await` hace por vos en código moderno — pero la rubrica del curso pide `completion` handlers.

13. Paso 2: las bajas y el 404

```

APIClient.shared.delete(recurso(apiId.int64Value)) { resultado in
    switch resultado {
    case .success:
        self.contexto.delete(entidad)
        self.borrarSiguiente(cola, recurso: recurso, completion: completion)
    case .failure(let error):
        // Un 404 significa que en el servidor ya no esta: el objetivo
        // ya se cumplio, asi que se borra la fila local igual. Sin esto
        // la baja quedaria trabada para siempre reintentando.
        if case .server(let status, _) = error, status == 404 {
            self.contexto.delete(entidad)
            self.borrarSiguiente(cola, recurso: recurso, completion: completion)
        } else {
            completion(.failure(error))
        }
    }
}
}

```

Un 404 al borrar es un éxito, no un error. Si el servidor dice "ese registro no existe", el objetivo ya esta cumplido: no existe. Tratarlo como fallo deja la baja trabada para siempre — cada reintento devuelve 404 otra vez. Este mismo criterio se repite en `ImagenService` al borrar fotos, donde antes causaba un bug real: la pantalla decia "Error del servidor" sobre una foto que **si** se habia borrado.

14. Paso 3: el upsert y sus dos defensas

"Upsert" es actualizar si existe, insertar si no. La busqueda es por `apiId`, que es el unico id que las dos puntas comparten:

`Sync/SyncManager.swift`

```

private func entidadParaUpsert<T: NSObject>(<
    _ tipo: T.Type, nombreEntidad: String, apiId: Int64
) -> T? {
    if let existente = buscarPorApiId(tipo, nombreEntidad: nombreEntidad, apiId: apiId) {
        let estadoSync = existente.value(forKey: "estadoSync") as? Int16 ?? 0
        let pendienteEliminar = existente.value(forKey: "pendienteEliminar") as? Bool ?? false
        guard estadoSync == 1, !pendienteEliminar else { return nil }
        return existente
    }

    let nueva = T(context: contexto)
    nueva.setValue(UUID().uuidString, forKey: "localId")
    nueva.setValue(NSNumber(value: apiId), forKey: "apiId")
    nueva.setValue(Int16(1), forKey: "estadoSync")
    nueva.setValue(false, forKey: "pendienteEliminar")
    return nueva
}

```

Ese `guard estadoSync == 1, !pendienteEliminar` es **la segunda linea de defensa** del offline. Aunque el orden de los pasos ya deberia garantizar que no queda nada pendiente, si por lo que fuera quedara, el upsert lo saltea (`return nil`) en vez de pisarlo.

Es defensa en profundidad: dos mecanismos independientes protegiendo el mismo dato.

Reconciliar borrados hechos desde afuera

```

/// Borra lo que alguien elimino desde el portal web. Sin esto, un producto
/// borrado en el servidor sobreviviria para siempre en el telefono.
///
/// Solo toca filas ya sincronizadas y con `apiId`: lo pendiente de subir no
/// se toca, y lo que nunca se subio no existe alla por definicion.
private func eliminarLocalesQueElServidorYaNoTiene(_ nombreEntidad: String,
                                                    apiIds: Set<Int64>) {
    let request = NSFetchRequest<NSManagedObject>(entityName: nombreEntidad)
    request.predicate = NSPredicate(format: "estadoSync == 1 AND apiId != nil")
    let locales = (try? contexto.fetch(request)) ?? []
    ...
}

```

Sin esto, un producto borrado desde el portal web seguiria apareciendo en el telefono para siempre. El predicado vuelve a proteger lo pendiente: solo mira filas ya sincronizadas.

15. Que ve el usuario cuando falla

Features/Productos/ProductoListViewModel.swift

```

func sincronizar() {
    onLoadingChanged?(true)
    SyncManager.shared.sincronizar { [weak self] resultado in
        guard let self else { return }
        self.onLoadingChanged?(false)
        switch resultado {
        case .success:
            self.cargarLocales()
        case .failure(let error):
            // Los datos locales siguen en pantalla: que falle el sync no
            // vacia la lista.
            self.onError?(error.errorDescription ?? "No se pudo sincronizar")
        }
    }
}

```

Una sincronizacion fallida muestra un mensaje y **nada mas**. La lista sigue como estaba, con sus datos locales. Es coherente con toda la arquitectura: la red es un extra, no una dependencia para funcionar.

PARTE VI

Proceso 5: fotos de producto

La unica parte de la app que no es offline-first, y por que.

16. Por que las fotos son online

Subir una foto necesita el apiId del producto: el backend la asocia al recurso por su id. Un producto creado sin conexion todavia no tiene ese id, asi que la operacion **no se puede encolar**.

En vez de dejar un boton que falla, la app **no lo muestra** y explica por que en pantalla. La limitacion se convierte en informacion.

El cuerpo multipart, armado a mano

```

/// Arma el cuerpo multipart a mano. El formato es sensible a los saltos de
/// linea: van CRLF (`\r\n`) y no `n`, y la ultima frontera lleva `--` al
/// final. Un solo salto de linea mal puesto hace que el servidor no
/// encuentre el archivo y conteste 400 sin explicar por que.
private static func cuerpoMultipart(...) -> Data {
    var cuerpo = Data()
    let salto = "\r\n"

    cuerpo.append("--\({frontera})\({salto}")
    cuerpo.append("Content-Disposition: form-data; name=\"\({campo})\"; filename=...")
    cuerpo.append("Content-Type: \({mimeType})\({salto}")
    cuerpo.append(salto)
    cuerpo.append(archivo)
    cuerpo.append(salto)
    cuerpo.append("--\({frontera})--\({salto}")

    return cuerpo
}

```

Es el unico lugar donde el cuerpo no es JSON. La **frontera** es un separador aleatorio que marca donde empieza y termina cada parte del formulario.

Las fotos no se guardan en el telefono

Se suben a **Cloudinary** y lo que queda local es la URL. Para mostrarlas hay un descargador con cache en memoria:

```

if let cacheada = DescargadorDeImagenes.shared.imagenCacheada(urlTexto) {
    mostrarFoto(cacheada)
    return
}

mostrarPlaceholder()
DescargadorDeImagenes.shared.descargar(urlTexto) { [weak self] imagen in
    guard let self, self.urlActual == urlTexto, let imagen else { return }
    self.mostrarFoto(imagen)
}

```

El guard que evita fotos cruzadas. `self.urlActual == urlTexto` es la trampa clasica de las listas. Las celdas se reusan al hacer scroll: si empezaste a bajar la foto del producto A y esa celda ya se reciclo para el producto B, pintar la imagen mostraria la foto de A en la fila de B. Comparar la URL antes de pintar es lo que lo evita.

PARTE VII

Proceso 6: roles y seguridad

Quien puede hacer que, y donde se decide de verdad.

17. Los tres roles

Rol	Productos	Categorías y proveedores	Usuarios
LECTOR	Solo ver	Solo ver	No accede
OPERADOR	Crear, editar, fotos	Solo ver	No accede
ADMIN	Todo, incluido borrar	Todo	Todo

Auth/SessionManager.swift

```

/// Spring Security no es consistente con el prefijo `ROLE_`: segun como este
/// armado el token, el mismo rol puede llegar como `ADMIN` o `ROLE_ADMIN`.
/// Se comparan las dos formas sin el prefijo para no depender de eso.
func hasRole(_ role: String) -> Bool {
    let buscado = Self.sinPrefijo(role)
    return roles.contains { Self.sinPrefijo($0) == buscado }
}

var esAdmin: Bool { hasRole("ADMIN") }

/// Crear/editar productos y manejar sus imagenes es de OPERADOR para arriba.
/// Se chequean los dos roles y no solo OPERADOR: no hay garantia de que el
/// backend le agregue OPERADOR a un ADMIN en el token.
var puedeEditarProductos: Bool { esAdmin || hasRole("OPERADOR") }

```

La normalizacion del prefijo no es paranoia: se confirmo leyendo el backend, donde CustomUserDetailsService arma las authorities como "ROLE_" + rol.name().

18. Esconder botones no es seguridad

La app usa esos flags para **ocultar** lo que no corresponde. Por ejemplo, el swipe para eliminar un producto:

Features/Productos/ProductoListViewController.swift

```

func tableView(_ tableView: UITableView,
               trailingSwipeActionsConfigurationForRowAt indexPath: IndexPath
) -> UISwipeActionsConfiguration? {
    // Borrar productos es de ADMIN, no de OPERADOR: es el unico verbo de la
    // tabla de roles de PLAN.md que pide mas que editar.
    guard SessionManager.shared.esAdmin else { return nil }
    ...
}

```

Esto es comodidad, no seguridad. Cualquiera puede llamar a la API por fuera de la app. Lo que realmente protege es el **backend**, que valida el rol en cada peticion y responde **403** si no corresponde. El gating de la app existe para no ofrecer botones que van a fallar — es buena experiencia de usuario, no una barrera.

19. Cuando el token vence

Un JWT tiene fecha de vencimiento. Cuando pasa, el backend empieza a responder 401 y la app tiene que reaccionar sola:

Auth/SessionManager.swift

```
/// Cierra la sesion y avisa a la UI. **El guard no es decorativo:** una
/// sincronizacion dispara varias requests seguidas y todas van a fallar con
/// 401 al vencer el token; sin el, se apilarian varios avisos encima del
/// mismo problema.
func manejarSesionExpirada() {
    guard isAuthenticated else { return }
    logout()
    NotificationCenter.default.post(name: Self.sesionExpirada, object: nil)
}
```

La notificacion la escucha el InicioTabBarController, y no cada pantalla. Es el controlador que el Login presento, asi que es el unico que puede volver atras de una sola vez, sin importar en que tab o en que formulario este parado el usuario.

PARTE VIII

Detalles transversales

Piezas chicas que aparecen en todos lados.

20. Las fechas del backend

Networking/FechaAPI.swift

```
/// `fechaRegistro` viene como `2026-08-12T10:15:30`: ISO 8601 **sin zona horaria
/// ni fracciones de segundo**. `JSONDecoder.dateDecodingStrategy = .iso8601` lo
/// rechaza justamente por no traer zona, y un decode fallido de una sola fecha
/// tumba el listado entero – por eso se parsea a mano.
enum FechaAPI {
    private static let formatos = [
        "yyyy-MM-dd'T'HH:mm:ss",
        "yyyy-MM-dd'T'HH:mm:ss.SSS",
        "yyyy-MM-dd'T'HH:mm:ss.SSSSSS"
    ]
}
```

Se prueban tres formatos porque el backend no es consistente. Y hay un detalle que arruina apps en produccion:

```
/// `en_US_POSIX` es obligatorio: sin eso el formato depende de la config
/// regional del telefono y el parseo falla en dispositivos con calendario o
/// idioma distintos.
formateador.locale = Locale(identifier: "en_US_POSIX")
```

El bug que no ves en tu telefono. Sin en_US_POSIX, un usuario con el calendario budista o japones configurado obtiene fechas que no parsean, y la lista entera queda vacia. En el simulador del desarrollador nunca pasa.

21. El sistema de diseno

Los colores, tipografias y espaciados no estan escritos en cada pantalla: viven en DesignSystem/Theme.swift, que traduce el documento de diseno a codigo.

Si cambia el diseño se toca ese archivo y nada más. Las pantallas piden `Theme.Color.charcoalDeep` o `Theme.TextStyle.bodyLG`, nunca un color ni un tamaño literal.

22. Donde encaja Firebase

Firebase guarda **una sola cosa**: la bitácora de auditoría. No participa de ninguno de los procesos anteriores — no toca Core Data, no pasa por el SyncManager y no usa APIClient.

Los seis puntos donde el usuario modifica datos llaman a `BitacoraService.registrar()` después de guardar. Está explicado en detalle en `Firebase-Implementacion.pdf`.

23. Resumen: donde mirar según que quieras entender

Si quieres entender...	Abre este archivo
Como arranca la app	<code>App/AppDelegate.swift</code>
Como se carga la base local	<code>CoreData/PersistenceController.swift</code>
Como se hace cualquier petición HTTP	<code>Networking/APIClient.swift</code>
Que endpoints existen	<code>Networking/Endpoint.swift</code>
Como se guarda la sesión	<code>Auth/SessionManager.swift</code> + <code>KeychainService.swift</code>
Como se lee de la base local	<code>Services/ProductoService.swift</code>
Como funciona el offline	<code>Sync/SyncManager.swift</code> (el archivo más importante)
Como se suben fotos	<code>Services/ImagenService.swift</code>
Como se decide que ve cada rol	<code>Auth/SessionManager.swift</code>
Como se ve una pantalla por dentro	<code>Features/Productos/</code> (VC + ViewModel + Cell)
Los colores y tipografías	<code>DesignSystem/Theme.swift</code>

Si vas a leer un solo archivo, que sea SyncManager. Ahí está condensada la lógica que da sentido a todo lo demás: por que las entidades tienen cuatro campos técnicos, por que el borrado es lógico, por que existe el chip naranja y por que el orden de las operaciones es el que es.